P2929R2
Proposal to add simd_invoke to std::simd

Draft Proposal, 2026-01-26

This version:
http://wg21.link/D2929R2.html

Authors:
Daniel Towner (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to extend `std::simd` with a method of allowing a lambda to be invoked on smaller pieces of a SIMD value in order to make interaction with intrinsics easier.

Table of Contents

§ 1. Motivation

ISO/IEC 19570:2018 introduced data-parallel types to the C++ Extensions for Parallelism TS [P1928R15]. That paper, and several ancillary papers, do an excellent job of setting out the main features of an extension to C++ which allows generic data parallel programming on arbitrary targets. However, it is inevitable that the programmer will want to make some use of target-specific intrinsics in order to unlock some of the more unusual features of those specific platforms. This requires that the programmer is able to allow a `basic_vec` value to be used in a call to a target intrinsic, and that the result of the intrinsic call can be used to generate a new `basic_vec` value. This is already permitted for `basic_vec` values which fit into a native register, but it is harder to achieve when the `basic_vec` value spans multiple registers.

In this paper we will propose a function called `simd::chunked_invoke` which makes it easy to repeatedly apply a target specific intrinsic to native-sized pieces of large `basic_vec` value arguments (created using `simd::chunk`), and to mar-

shall their individual results back into a `basic_vec` result (using `simd::cat`).

The function is named `chunked_invoke` and placed in `std::simd` to directly align with related established functions such as `chunk` and `cat`. This terminology was suggested by committee members during prior reviews, and reflects the internal vocabulary of the SIMD library."

2. Revision History

R1 => R2

- Updated to match the Working Draft.
- Changed the name of the function to avoid confusion with `std::invoke`.
- Defined callable order to be by ascending index.
- Numerous minor clarifications and wording fixes.
- Changed the Constraints into Mandates to make them hard errors.
- Added description of why prototype-based chunking is not supported.

R0 => R1

- Freshened up the wording to match the current state of the draft proposal.
- Removed `invoke_indexed` in favour of probing the callables capabilities.

3. Background

Although `std:simd` has been carefully crafted to include APIs which access all of the common or desirable features of SIMD instruction sets, it is inevitable that the user will sometimes want to take advantage of instructions which are specific to a particular platform. For example, a DSP target may have a special type of accumulator instruction, or algorithm specific instruction (e.g., AES crypto). Clearly, calling these intrinsics results in non-portable code, but the increase in hardware-accelerated performance on a given target could be a worthwhile trade-off.

The draft standard of `std::simd` recommends that provision is made for conversions to and from implementation defined types. For example:

```
vec<float>
addsub(vec<float> a, vec<float> b) {
  return static_cast<vec<float>>(_mm256_addsub_ps(static_cast<__m256>(a),
                                                  static_cast<__m256>(b)));
}
```

In this example the inputs, which are native register-sized `basic_vec` values, are explicitly converted into their target-specific typed values. Those target specific types are used to call the intrinsics, and then the target specific return value is converted back into the closest `basic_vec` representation.

This example is straightforward, since the `basic_vec` values are explicitly the correct native size. For targets which support several different register sizes (e.g., some variants of AVX support 128-, 256- and 512-bit registers) the code can use a constexpr conditional to select which size to use:

```
vec<float>
addsub(vec<float> a, vec<float> b) {
  if constexpr (vec<float>::size() == 4)
    return vec<float>(_mm_addsub_ps(static_cast<__m128>(a), static_cast<__m128>(b)));
  else if constexpr (vec<float>::size() == 8)
    return vec<float>(_mm256_addsub_ps(static_cast<__m256>(a), static_cast<__m256>(b)));
```

```
  else
    error(); // Invalid native register
}
```

Things become more tricky when dealing with `basic_vec` values which are larger than their implementation types. Such types cannot be converted into a type which can be used to call an intrinsic. Instead, the `basic_vec` must be broken down into small pieces which are the correct size for the call to the intrinsic, and then the results of that glued back together. Here is one way to do that for a `basic_vec` value which is twice as big as a native register:

```
// Assumes AVX is in use, and that each native register is therefore 8xfloat
vec<float>
addsub(vec<float, 16> a, vec<float, 16> b) {

  // Get register-sized pieces
  auto {lowA, highA} = chunk<vec<float>>(a);
  auto {lowB, highB} = chunk<vec<float>>(b);

  // Call the intrinsic on each pair of pieces.
  auto resultLow = vec<float>(_mm256_addsub_ps(static_cast<__m256>(lowA),
                                               static_cast<__m256>(lowB)));
  auto resultHigh = vec<float>(_mm256_addsub_ps(static_cast<__m256>(highA),
                                                static_cast<__m256>(highB)));

  // Glue the individual results back together.
  return cat(resultLow, resultHigh);
}
```

This is now getting verbose, and it only handles `basic_vec` value inputs which are twice the size of a register value. To use the intrinsic with larger `basic_vec` values, or `basic_vec` values which don't map into native register-sized pieces, more work is needed (e.g., if `vec<float, 20>` was used then the pieces would be of size 8, 8, and 4 respectively, and this would require a suitable call to the intrinsic of the appropriate size).

The boiler-plate code needed to handle this is technically straight-forward, but verbose. A completely generic solution which could handle arbitrary `basic_vec` value size would also require additional mechanisms like immediately invoked lambdas or index sequences to be used too. Rather than requiring every user to have to write their own intrinsic call handlers, we can abstract the general mechanism into something that is easily reused. In particular we want to break a set of `basic_vec` value arguments into smaller pieces, call an intrinsic on each, and then glue the results of those intrinsic calls back together. We achieve this though a proposed function called `simd::chunked_invoke` which we will describe in the remainder of this paper.

§ 4. Description of `simd::chunked_invoke`

The `simd::chunked_invoke` function is rather like the standard `invoke` in that it takes a callable object and a set of arguments. Its basic signature is as follows:

```
template<typename Fn, typename... Args>
auto simd::chunked_invoke(Fn fn, Args...);
```

The `fn` parameter should be a callable that accepts some arguments which can be used to invoke an intrinsic from a native register. For example, to continue our example from above, we could create a utility function which calls the `_mm256_addsub_ps` intrinsic from native-sized `basic_vec` values:

```
inline auto native_addsub(vec<float> lhs, vec<float> rhs) {
    auto nativeLhs = static_cast<__m256>(lhs);
    auto nativeRhs = static_cast<__m256>(rhs);
```

```
    return vec<float>(_mm256_addsub_ps(nativeLhs, nativeRhs));
}
```

Given this wrapper for native-sized intrinsic calls, we can now use `simd::chunked_invoke` to break down a large `basic_vec` value into individual register-sized calls:

```
auto addsub(vec<float, 32> x, vec<float, 32> y)
{
    return simd::chunked_invoke(native_addsub, x, y);
}
```

The `simd::chunked_invoke` function accepts any number of arguments and will break each one down into native-sized in a way which is equivalent to calling `simd::chunk` on each argument. Respective chunked pieces of each argument are then used to invoke the supplied function argument. On completion, the individual results are glued back together using `simd::cat` to produce a single `basic_vec`/`basic_mask` value result.

Let's look how the example from § 3 Background looks like with `chunked_invoked` applied using before-after table:

Before

```
auto addsub(vec<float, 16> a, vec<float, 16> b) {

    auto [lowA, highA] = chunk<vec<float>>(a);
    auto [lowB, highB] = chunk<vec<float>>(b);

    auto resultLow = vec<float>(_mm256_addsub_ps(
            static_cast<__m256>(lowA),
            static_cast<__m256>(lowB)));
    auto resultHigh = vec<float>(_mm256_addsub_ps(
            static_cast<__m256>(highA),
            static_cast<__m256>(highB)));

    return cat(resultLow, resultHigh);
}
```

After

```
auto addsub(vec<float, 16> a, vec<float, 16> b)
{
    auto do_native = [](vec<float> lhs, vec<floa
        return vec<float>(_mm256_addsub_ps(
            static_cast<__m256>(lhs),
            static_cast<__m256>(rhs)));
    };

    return chunked_invoke(do_native, a, b);
}
```

By default the function will use the native size for the element type, with the aim of calling the intrinsic with the largest permitted builtin type. However, `simd::chunked_invoke` can also be explicitly given the size of block to use. For example, the following code breaks down into pieces of size 4 instead (and also supplies the callable as a local lambda, to make the function self-contained):

```
auto addsub(vec<float, 32> x, vec<float, 32> y)
{
    auto do_native = [](vec<float, 4> lhs, vec<float, 4> rhs) {
        return vec<float, 4>(_mm_addsub_ps(static_cast<__m128>(lhs),
                                           static_cast<__m128>(rhs)));
    };

    return simd::chunked_invoke<4>(do_native, x, y);
}
```

Being able to define a different size is useful for two reasons:

1. We may wish to process data in smaller input sizes than native size. For example, if the data needs to be upconverted to a large element size for the operation, then it can be useful to choose a smaller block size to begin with so that the upconverted data is no larger than a native register. This could be more efficient than accepting native register-sized data and then upconverting to several registers.
2. If the element type of the arguments are different then the `simd::chunked_invoke` function cannot determine the appropriate native size for itself. For example, if the first argument has `float` elements and the second argument had

`int8_t` elements then the number of elements to use in the calls cannot be inferred, and the call to `simd::chunked_invoke` must be told how many elements to use in each block.

So far we have only considered what happens when `simd::chunked_invoke` is given `basic_vec` value arguments which are multiples of some native register size, but as we saw in our introduction, it might be useful to be able to call `simd::chunked_invoke` on `basic_vec` values of arbitrary size. For example, suppose that the add/sub is being called on a `basic_vec` value with 19 elements. In that case on a target with a native size of 8 elements `simd::chunked_invoke` would need to break the calls down into pieces of sizes 8, 8, and 3 respectively, and the callable would need to be able to handle `basic_vec` values of arbitrary size. The following example shows how this might work:

```
auto addsub(vec<float, 19> x, vec<float, 19> y)
{
    // Invoke the most appropriate intrinsic for the given simd types.
    auto do_native =
      []<typename T, typename ABI>(basic_vec<T, ABI> lhs, basic_vec<T, ABI> rhs) {
        if constexpr (basic_vec<T, ABI>::size <= 4)
            return vec<float, 4>(_mm_addsub_ps(static_cast<__m128>(lhs),
                                               static_cast<__m128>(rhs)));
        else
            return vec<float, 8>(_mm256_addsub_ps(static_cast<__m256>(lhs),
                                                  static_cast<__m256>(rhs)));
    };

    return chunked_invoke(do_native, x, y);
}
```

In this example the local lambda function can accept `basic_vec` inputs of any size, and will choose the most appropriate intrinsic to use. For example, given the block of 3 tail elements the lambda utility will convert the 3 elements to an `__m128` register and call `_mm_addsub_ps`. This lambda function can then be called by `chunked_invoke` to enable an arbitrarily sized set of `basic_vec` arguments to be mapped onto their underlying intrinsics. For the example above, the following code was generated:

```
vmovups ymm0, ymmword ptr [rsi]
vmovups ymm1, ymmword ptr [rsi + 32]
vmovups xmm2, xmmword ptr [rsi + 64]
mov     rax, rdi
vaddsubps       ymm0, ymm0, ymmword ptr [rdx]
vaddsubps       ymm1, ymm1, ymmword ptr [rdx + 32]
vaddsubps       xmm2, xmm2, xmmword ptr [rdx + 64]
vmovups ymmword ptr [rdi], ymm0
vmovups ymmword ptr [rdi + 32], ymm1
vmovups xmmword ptr [rdi + 64], xmm2
```

Notice how the load, addsub and store instructions work respectively in ymm, ymm and xmm registers to cope with the different sizes.

Having now described the basic operation of `simd::chunked_invoke` we can consider some of the rules for using it, and a useful extension to it which makes certain scenarios easier to deal with.

§ 4.1. Using indexed Callable invocations

When a large `basic_vec` value is broken down into pieces to invoke the callable function, the size of that piece can be obtained from the callable function invocation's parameter type but it can also be useful to pass in the index of that piece too. For example, suppose a function is invoking an intrinsic to perform a special type of memory store operation. Each register-

sized sub-piece of the incoming `basic_vec` needs to know its offset so that it can be written to the correct pointer offset. The following example code illustrates how this could happen:

```
auto special_memory_store(vec<float, 32> x, float* ptr)
{
    // Invoke the most appropriate intrinsic for the given simd types.
    auto do_native =
      [=]<typename T, typename ABI>(basic_vec<T, ABI> data, auto idx) {
        (_mm256_special_store_ps(ptr + idx,   // NEED TO USE THE OFFSET HERE
                                 static_cast<__m256>(data)));
      };

    chunked_invoke(do_native, x);
}
```

The `invoke` function can probe the Callable that it is given to determine if it will accept an offset as its last parameter. If the extra parameter can be accepted then the offset of the sub-piece within the parent `basic_vec` will be passed in too (i.e., 0, 8, 16 and 24 in this example).

Note in this example that the Callable does not return a value itself as it is writing to memory.

4.1.1. Design option - avoiding probing the index capabilities

In the first revision of this paper we proposed that the function which invokes a Callable with an explicit offset should be called with a `_indexed` suffix (e.g., `simd::chunked_invoke_indexed`). This name makes it clear that it expects the Callable to have the extra parameter. However, a precedent has been set in the `simd::permute` function to allow Callables to be probed for their capabilities rather than naming the function to call it out (e.g., `permute` can optionally take a `size` parameter), so we have now followed suit here.

4.2. Considerations in using `simd::chunked_invoke`

These are a set of considerations for using `simd::chunked_invoke`. In the following, `simd-vec-or-mask-type` is a type that is either a `basic_vec` or a `basic_mask`.

All the arguments that are passed to the callable function must satisfy `simd-vec-or-mask-type`. The arguments can be a mixture of `basic_vec` or `basic_mask` values.

The chunk size template parameter `N` is forwarded from `simd::chunked_invoke` to `simd::chunk` to decompose each argument into appropriately sized pieces. The chunk sizes for each argument are exactly the same as those produced by `simd::chunk<N>`, including the handling of tail (remainder) elements.

Unlike `simd::chunk` which offers a prototype parameter to control chunking ABI (e.g., to force chunks into a specific ABI type), `simd::chunked_invoke` intentionally does not provide this feature. In practice, the chunking performed by `simd::chunked_invoke<N>` automatically preserves the ABI of each input argument already, as chunk types are formed using `resize_t<NewSize>(old_chunk)`, thus retaining their original ABI. Supporting prototype-based chunking for multiple arguments would require users to specify a prototype for each argument, which is both complicated and error-prone. As a convenience utility, `simd::chunked_invoke` is intentionally limited to common, safe use cases. For advanced scenarios that demand explicit ABI control or distinct chunking strategies per argument, users should write custom, case-specific code rather than rely on this facility.

When multiple `simd-vec-or-mask-type` arguments are provided, each must have the same number of elements. This ensures that for each argument simd::chunk produces the same number of chunks, with corresponding chunks having matching sizes.

For the native block size to be deduced, all the `simd-vec-or-mask-type` objects must have the same native size. This is to ensure that when the `simd-vec-or-mask-type` arguments are broken into pieces they will always map to the same respective sizes. If the size cannot be deduced like this then the user must explicitly supply the block size.

The callable fn provided to `simd::chunked_invoke` must be able to accept chunk arguments of any size produced during decomposition, including smaller tail chunks when the total size is not a multiple of the chunk size. Failure to support all possible chunk sizes will result in ill-formed code or undefined behavior.

The callable order is defined to be in increasing order of chunk index (i.e., from [0..NumChunks)) to allow the invoked function to accumulate state in a predictable way.

When the Callable returns a value, it must return a `simd-vec-or-mask-type` object. This is because there is no way to take non-`simd-vec-or-mask-type` results from the Callable and merge them together, except by using `cat`.

When the Callable returns a `simd-vec-or-mask-type` object, it need not have the same size as its input arguments. For example, the Callable function could perform an operation like extracting multiples of some index, where the results have to be concatenated back together.

When the Callable returns a `simd-vec-or-mask-type` object, every invocation of the callable must return a `simd-vec-or-mask-type` object with the same element type. This is to ensure that the results can be glued together.

§ 5. Wording

The wording diff is against the current C++ Working Draft.

§ 5.1. Modify [**simd.syn**]

Insert new exposition only concepts after *simd-type*:

```
template<class V>
  concept simd-mask-type =                                          // exposition only
    same_as<V, basic_mask<mask-element-size<V>, typename V::abi_type>> &&
    is_default_constructible_v<V>;


template<class V>
  concept simd-vec-or-mask-type =                                   // exposition only
    simd-vec-type<V> || simd-mask-type<V>;
```

§ 5.2. Add a new section to [**simd.syn**]

```
  template<simd-floating-point V>
    rebind_t<complex<typename V::value_type>, V> polar(const V& x, const V& y = {});

  template<simd-complex V> constexpr V pow(const V& x, const V& y);

  // [simd.chunked.invoke] chunked_invoke utility function
  template<simd-size-type N = see below, class Fn,
           simd-vec-or-mask-type Arg0, simd-vec-or-mask-type... Args>
    constexpr auto chunked_invoke(Fn fn, Arg0 first_arg, Args... other_args);
```

§ 5.3. Add new section [**simd.chunked.invoke**]

`chunked_invoke` utility function **[simd.chunked.invoke]**

```
template<simd-size-type N = see below, class Fn,
         simd-vec-or-mask-type Arg0, simd-vec-or-mask-type... Args>
  constexpr auto chunked_invoke(Fn fn, Arg0 first_arg, Args... other_args);
```

Let:

- `N` be set to `simd::vec<typename Arg0::value_type>::size()` if the caller does not provide a value for `N`.
- `NumChunks` be the number of tuple elements in the result of calling `chunk<N>(first_arg)`.
- `ArgChunks(A, i)` be a function which returns the `i`th element of `chunk<N>(A)`, with `i` in the range `[0..NumChunks)`.

Mandates:

- `((Arg0::size() == Args::size()) && ...)` is `true`.
- The result type of `fn` is `void` or satisfies `simd-vec-or-mask-type`.
- The callable `fn` shall be invocable for every combination of chunk argument types that may be produced, including tail chunks of sizes less than `N`.

Effects:

- For each *i* in the range `[0..NumChunks)`, the Callable function `fn` is called with the following arguments:
  - `ArgChunks(first_arg, i)` for the first argument, and
  - `ArgChunks(other_args, i)` for each of the other arguments.
  - If `fn` is a Callable which is well-formed when given an additional parameter of type `simd-size-type`, then the last parameter will be set to the compile-time constant value `i * N`, otherwise `fn` will be called without that extra parameter.
- [Note: If the callable `fn` is invocable with and without the chunk index, the form accepting the index as an additional trailing argument is selected. Care should be taken to avoid ambiguous overloads or call signatures. — end note]
- If `fn` has a non-void return type, then `Result` is a tuple of `NumChunks` elements, where the *i*th element is the result of the *i*th call to `fn` as described above.
- If `fn` has a void return type, then `Result` is `void`.

Remarks:

- `fn` is invoked exactly once for each chunk index *i*, in increasing order of *i*.
- If the chunk size does not divide the argument size exactly, the last chunk may be smaller. This mirrors the behavior of `simd::chunk`.

Returns:

If the Callable function has a void return, return nothing, otherwise return `cat(Result)`.